Sudharshan Hegde

sudharshanhegde68@gmail.com

sudharshan_hegde:matrix.org

OpenSSH - compatible CLI

DESCRIPTION

libssh is production grade C library which implements SSH-2 protocol which is used by projects such as KDE and QEMU,but it ships no production-quality command-line client, only example binaries in **examples/** explicitly marked as API demonstrations. Recent contributions have closed some gaps : **-o key=Value** CLI parsing, **match/originalhost** which was added. However, of the 56 directives that currently map to SOC_NA or SOC_UNSUPPORTED in src/config.c, approximately 20 that are directly needed for a functional CLI such as LocalForward, RemoteForward, DynamicForward, RequestTTY, ForwardX11, ServerAliveInterval, PreferredAuthentications among others are silently discarded, and the existing ssh_client.c example has no PTY handling, no forwarding, no escape sequences, and no keepalive.

This project will deliver two things :

- Promoting the ignored directives to first class library options in **src/config.c**, **src/options.c** and **include/libssh/session.h** so that any libssh application can use them.
- New ssh binary in tools/ssh/ that uses options to provide an interactive shell with PTY and escape sequence support, all forwarding modes (**-L**, **-R**, **-D**, **-A**, **-X/ -Y**), and SSH keepalive which will be behaviorally compatible with openSSH's ssh(1) . Connection multiplexing via controlMaster/controlPath is a stretch goal if the time permits.

Mentor : Jakub Jelen

What city and country will you reside in during the summer?

City : Bengaluru,Karnataka.

Country : India.

Will the proposed work address an open issue at the libssh gitlab? If yes, paste the link to the issue.

libssh/libssh-mirror#297

What benefits does your proposed work have for libssh and its community?

The work delivers two levels simultaneously. At library level, roughly 20 SSH config directives such as **LocalForward**, **RemoteForward**, **DynamicForward**, **RequestTTY, ForwardX11, ServerAliveInterval, PreferredAuthentications** among others which currently map to **SOC_NA** or **SOC_UNSUPPORTED** in **src/config.c** are silently discarded. Any application built on libssh, not just a CLI will inherit this gap. A developer using libssh in their application who writes these directives in **~/.ssh/config** gets no error and no effect. Promoting them to first-class SSH_OPTIONS_* constants will fix this for the entire libssh ecosystem and every downstream project will benefit without changing a line of their own code.

At CLI level distribution that ship libssh currently has no ssh binary to package alongside it. A production quality binary in tools/ssh/ that will be behaviourally compatible with openSSH's ssh(1) gives distributions something shippable and gives developers evaluating SSH libraries a direct comparison point. It also addresses the open Issue libssh/libssh-mirror#297.

Building a CLI that exercises a full library end to end, PTY , forwarding,AUTH, Keepalive

will inherently stress test code paths that unit tests and example binary never reach, which will help surface integration issues before it affects production users.

Why are you the right person to work on this project?

I do have two active merge requests in libssh currently under review with the mentor, Jakub Jelen.

- 774 : This merge request addressed the issue of outstanding doc issues with no proper doxygen documentation where I had to read internal functions in socket.c, options.c, session.c, threads.c, poll.c, misc.c, buffer.c and error.c which are also the files which needs modification in the project, So documentation work doubled as codebase orientation.
- 743 : This merge request deprecates SCP support behind a **with_scp** Cmake Build option, it will gate SCP implementation, headers, examples and tests, also it adds new public API ssh_scp_is_closed() to detect channel closure during transfer.

I also approach this project as a user. While experimenting with a Large language model, I ran a local version on my laptop which I had configured with a custom system prompt for my specific use case, to access the model which was configured for my requirement from any of my devices, I used SSH. This gave me a direct understanding of SSH from the user side, not just as a developer reading the spec, but as someone who depends on it working correctly.

On technical side :

- I have implemented a peer to peer file transfer project which implements a tracker - based file chunks with central tracker, discovering each other, and downloading chunks in parallel over TCP with JSON protocol giving me practical experience with connection lifecycle, TCP and concurrent I/O. (github link)
- I also had implemented a mini SQL database of my own written in C++, which implements its own parser,storage engine,query planner and transaction

manager from scratch. While implementing this particular project I learnt a lot about the C/C++ system end to end.(github link)

- I also happen to be an engineering student currently pursuing my bachelors in Computer Science and engineering, where I was exposed to subjects such as Operating system, Computer Organization and Architecture, Cryptography and Network systems, Computer Networks and yes lot of maths (Math - 1, Math - 2, Math -3, Math - 4) Ranging from trigonometry to probability. During the coursework my favorite subject used to be maths and I was able to see its direct usage in cryptography and network systems which particularly drew me towards the subject which later introduced me towards SSH which implements it.
- Aside from this I am a keen learner who loves to solve problem one which I face and one which many like me face for example : I recently had to go through large sql views and stored procedures and it was difficult for me to create a flow chart of how the data was moving so I built something of my own which will make the visualization easy and solves the problem.(github link)
  I do have potential to work on this project along with the background and requirement as I have worked on few issues which makes me familiar with the codebase and also the functional requirement.

Is your proposal for a medium (175h) or big (350h) project?

Big (350h).

How do you plan to achieve completion of your project?

March 16 - March 31 (Application Period) :

- Both active merge requests (SCP deprecation, Doxygen documentation) are under review with the mentor. I will incorporate with review feedback and work towards getting them merged.
- I will submit a small pre-proposal patch fixing dead code bug in src/config.c where controlmaster maps to **SOC_NA** in keyword table but a full case

SOC_CONTROLMASTER: parsing block exists unreachably in the switch, patch will make them both consistent.

- I will continue refining this proposal with mentor feedback.

March 31 - April 30 (Acceptance Waiting Period) :

- I might have my final semester exams during this period for a week which will not significantly affect progress.
- I will finalize the API design for around 20 new SSH_OPTIONS_* constants - with exact field types,naming conventions and how the forwarding specs (**[bind_addr]port:host:hostport**) will be stored in **session -> opts**.
- I will study OpenSSH's ssh(1) behaviour for each feature being implemented to ensure the compatibility particularly around **RequestTTY** auto/yes/no/force logic. **PreferredAuthentications** ordering and escape sequence handling.
- I will set up the tools/ssh/ directory skeleton and CMakeLists.txt locally to validate the build structure before coding begins.

May 1 - May 24 (Community Bonding Period)

- I will discuss the implementation plan with mentor Jakub Jelen, agree on the exact list of options to promote, and settle on an incremental MR strategy — one feature area per MR, each with tests, to keep review load manageable.
- I will study libssh's torture test infrastructure to understand how to write config and option unit tests consistent with existing test patterns.
- By the end of this period I will have a clear, mentor-approved roadmap and a working skeleton binary that connects to a server and authenticates, confirming the build infrastructure is correct before the main coding begins

May 25 - July 10 (Library Layer)

Week 1 (May 25 - May 31) : Auth-Related options

- Promote batchmode, preferredauthentications, numberofpasswordprompts , each requires changes to session.h, libssh.h, config.c, options.c, and session.c.
- Write unit tests for each in tests/unittests/torture_options.c and torture_config.c.

Week 2 (June 1 - June 7) : Shell/PTY - related options

- Promote requesttty, sendenv, escapechar.
- Write unit tests.

Week 3 (June 8 - June 14) : Forwarding spec options

- Promote localforward, remoteforward, dynamicforward, these require a forwarding spec parser for [bind_addr:]port:host:hostport format.
- Write unit tests covering valid and malformed specs.

Week 4 (June 15 - June 21) : Forwarding control + keepalive options

- Promote exitonforwardfailure, gatewayports, serveraliveinterval, serveralivecountmax.
- Write unit tests.

Week 5 (June 22 - June 28) : X11 and agent options

- Promote forwardx11, forwardx11trusted, xauthlocation, forwardagent.
- Write unit tests.

Week 6 (June 29 - July 5) : Remaining options and review buffer

- Promote tcpkeepalive, connectionattempts, clearallforwardings.
- Buffer time for incorporating review feedback on earlier MRs from this phase.

- All ~20 library option MRs submitted.

Midterm Evaluation (July 10)

Milestone: All ~20 library options promoted to first-class SSH_OPTIONS_* constants, unit tests passing, MRs submitted for mentor review. The library layer is complete and any libssh application can now use these options through ssh_options_set() or ~/.ssh/config.

July 10 – August 3 (CLI Core)

Week 7 (July 10 - July 19) : tools/ssh/ argument parsing and session setup

- Full argument parsing: -p, -l, -i, -F, -o, -v, -N, -n, -J, -L, -R, -D, -A, -X, -Y and others.
- Complete auth chain using the newly promoted options — PreferredAuthentications ordering, BatchMode skipping interactive prompts, NumberOfPasswordPrompts counter.
- Basic non-interactive command execution confirmed working.

Week 8 (July 20 - July 26) : PTY and Interactive shell

- ssh_channel_request_pty(), cfmakeraw() for raw terminal mode.
- SIGWINCH handler with deferred flag, ioctl(TIOCGWINSZ) for window resize propagation.
- RequestTTY auto/yes/no/force logic.
- SendEnv via ssh_channel_request_env().
- Escape sequence state machine: ~. disconnect, ~^Z suspend, ~# list forwarded connections, ~? help, ~~ literal tilde.

Week 9 (July 27 - Aug 2) : Keepalive and integration buffer

- Server keepalive: background thread with pipe trick for main-thread safety, ServerAliveInterval/ServerAliveCountMax enforcement.
- Buffer for integration issues and review feedback.

August 3 – August 24 (Forwarding)

Week 10 (Aug 3 – Aug 9): Local port forwarding (-L)

- Listener socket with SO_REUSEADDR, ssh_event_add_fd() accept callback.
- On accept: ssh_channel_open_forward() to target, bridge the two with ssh_connector.
- ExitOnForwardFailure and GatewayPorts handling.

Week 11 (Aug 10–16): Remote port forwarding (-R) and dynamic forwarding (-D)

- -R: ssh_channel_listen_forward() global request, forwarded-tcpip channel callback, connect to local target.
- -D: SOCKS5 state machine — method negotiation, connection request parsing, ATYP handling for IPv4/domain/IPv6, bridge to ssh_channel_open_forward().

Week 12 (Aug 17–Aug 23): Agent forwarding (-A) and X11 forwarding (-X/-Y)

- -A: Unix domain socket to $SSH_AUTH_SOCK, auth-agent@openssh.com channel type, bridge to local agent.
- -X/-Y: fake cookie injection for untrusted forwarding, ssh_channel_request_x11(), connect incoming X11 channels to local display.
- Integration tests for all forwarding modes.

Final Evaluation (Aug 24 - Aug 31)

Milestone: Production-quality ssh binary in tools/ssh/ with interactive shell, PTY, escape sequences, all forwarding modes, agent forwarding, X11 forwarding, keepalive and behaviorally compatible with OpenSSH's ssh(1). All changes will be documented. MRs will be submitted for final review.

Stretch Goal (if time permits within the period)

- Connection multiplexing via ControlMaster/ControlPath: Unix domain socket control path, mux protocol (MUX_MSG_HELLO, MUX_C_NEW_SESSION), and connection reuse over existing master. This will only be attempted after all committed deliverables are complete and reviewed.

What are your past experiences with the open source world as a user and as a contributor?

As a user, open source software has been foundational to how I work. I develop on Linux daily and rely on tools like Git across all my projects, using these tools built my understanding of how open source projects are structured and maintained. libssh is my first project as a contributor. Through it I have learned the full development workflow, writing against existing conventions, navigating CI pipelines, responding to reviewer feedback, and understanding how a change propagates across headers, source files, build system, and symbol maps simultaneously.

Please include links to your code contributions which have already been merged, or to Gitlab merge requests for the issues you fixed for the project. This demonstrates your willingness to learn and familiarity with development workflow.

Contributions :

MR 743 : Deprecate SCP support behind with_scp CMake build option, add ssh_scp_is_closed() public API with unit tests.

MR 774 : Fix outstanding doxygen documentation warning across socket.c, options.c, session.c, poll.c, buffer.c, threads.c, error.c, misc.c and public API headers.

If available, please include links to any code you wrote for other open source projects.

I do not have contributions to other open source projects at this time. libssh is my first open source contribution.

What other relevant projects have you worked on previously and what knowledge have you gained from working on them?

Peer to Peer File Transfer System : A tracker-based chunk distribution system where peers register file chunks, discover each other, and download in parallel over TCP with a JSON protocol. Building this taught me how connection lifecycle works in practice and handling partial reads, managing concurrent connections with a thread pool, and designing a simple application-level protocol over raw sockets.
Mini SQL Database : A C++ implementation with its own parser, storage engine, query planner, and transaction manager. Working through this taught me how to structure a large C/C++ system from scratch and separating concerns across files, managing memory explicitly, and building incrementally.
CourseWork : Operating Systems, Computer Networks, and Cryptography and Network Systems gave me the theoretical foundation that underlies this project: process and I/O models, TCP/IP stack behaviour, and the cryptographic primitives SSH is built on.

What other time commitments, such as school work, exams, research, another job, planned vacation, etc? What are the dates for these commitments and how many hours a week do these commitments take?

I am currently in my final semester of engineering and interning at a company. I have final semester exams for approximately a week in April or May, which fall within the acceptance waiting period and will not significantly affect progress.
Regarding the internship, I plan to leave it before the coding period begins on May 25 in order to commit fully to this project. During the application and community bonding period (March–June), the internship may limit me to approximately 30-35 hours per week on GSoC-related preparation, but from May 25 onwards I will have no competing commitments and can dedicate full time to the project.

I have no planned vacations or other research commitments during the June–September coding period.